

Functional Programming

Radu Răzvan Slăvescu

Technical University of Cluj-Napoca
Department of Computer Science

Outline

Function Types

Recursive Types

Expression evaluation

- Call-by-value

- Call-by-name / by-need

List Processing with Recursion and Pattern Matching

- Building lists

- Basic functions on lists

- Association lists

Function Types and curried functions

Definition

We write $T1 \rightarrow T2$ to denote the type of all functions which map an argument of type $T1$ to a result of type $T2$.

Example

`not :: Bool -> Bool`

Remark

So, functions could also be regarded as types.

Function Types and curried functions

Example (Argument: **tuple** of Int; return: Int)

```
add :: (Int, Int) -> Int
add (x, y) = x + y
```

Example (Argument: Int; return: **function** from Int to Int)

```
add' :: Int -> (Int -> Int)
add' x y = x + y
```

Definition

A function is **curried** if it takes its arguments one at a time.

Remark

By default, every Haskell function is curried (like `add'`).

Curried functions - why are they useful?

Remark

Curried functions allow to easily and concisely define some useful functions by partially applying them.

Example (A function for incrementing numbers)

```
(add' 1)
```

```
Main> (add' 1) 2
```

```
3
```

Sections

Definition

If $\odot$ is an infix binary operator, then $(\odot)$, $(x\odot)$ and $(\odot y)$ are legal Haskell functions called **sections**.

Example (Guess)

```
(/2) 4.0
```

```
(2/) 4.0
```

```
:type (/)
```

Argument order and the arrow operator

Remark

To avoid excess parentheses, we state, by convention, that the arrow operator in types ($\rightarrow$) is right associative.

Example

`Int -> Double -> Char -> Bool` means
`Int -> (Double -> (Char -> Bool))`

Remark

Hence: applying curried functions is left associative.

Example

`f i d c` means `((f i) d) c`

Tree data type

Example (Type Tree in Haskell, using 3 constructors)

```
data Tree = Empty
         | Leaf Int
         | Node Tree Int Tree
```

```
t2 :: Tree
```

```
t2 = Node Empty 1 (Leaf 2)
```

Remarks

1. constructor functions do not have any defining equations; they just build data
2. recursivity is allowed in defining new types

Tree data type

Example (Type Tree in Haskell, using 3 constructors)

```
data Tree = Empty
          | Leaf Int
          | Node Tree Int Tree
```

Example (Constructor types and arities)

```
Empty :: Tree    (arity 0)
Leaf  :: Int -> Tree    (arity 1)
Node  :: Tree -> Int -> Tree -> Tree    (arity 3)
```

Remarks

1. **Constructor arity**: number of its arguments
2. **Type arity**: number of constructors in type definition

Expression evaluation

Question:

When does an expression get evaluated? ASAP or ALAP?

Question:

When calling a function, when are the arguments evaluated: before or after they replace the parameters?

Example (Square a number)

```
sq : number -> number
```

```
sq n = n*n
```

Example (Zero)

```
zeroit : number -> number
```

```
zeroit n = 0
```

Call-by-value

Definition

In case of "call-by-value", the **value** of the argument is sent.

Using call-by-value

Call-by-value is used in Elm. Elm is eager.

Call-by-value

Example

`sq (sq (sq 2))` =

`sq (sq (2 * 2))` =

`sq (sq 4)` =

`sq (4 * 4)` =

`sq 16` =

`16 * 16` =

256

Call-by-value

Example

`zeroit (sq (sq (sq 2))) =`

`zeroit (sq (sq (2 * 2))) =`

`zeroit (sq (sq 4)) =`

`zeroit (sq (4 * 4)) =`

`zeroit (sq 16) =`

`zeroit (16 * 16) =`

`zeroit 256 = 0`

Call-by-name

Definition

The **expression** of the argument is sent.

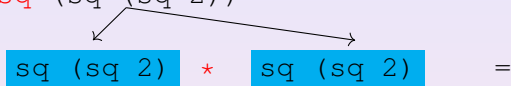
Example

```
zeroit (sq (sq (sq 2))) = 0
```

Call-by-name

Example

`sq (sq (sq 2))` =

 `sq (sq 2) * sq (sq 2)` =

`(sq 2 * sq 2) * sq (sq 2)` =

`((2*2) * sq 2) * sq (sq 2)` =

`(4 * sq 2) * sq (sq 2)` =

`(4 * (2 * 2)) * sq (sq 2)` =

...

Call-by-need

Definition

Call-by-need ("lazy evaluation") is similar to call-by-name, but evaluates an expression **only if necessary and only once**, then **shares** the result.

Use of call-by-need

Haskell uses call-by-need. Haskell is lazy.

Mechanism

$[x=E]$ means that every occurrence of x in an expression has access to E . If E is evaluated, it is replaced by its value. E is called **thunk**.

Call-by-need

Example

`sq (sq (sq 2))` =

`z * z [z=sq (sq 2)]` =

`z * z [z=y * y] [y=sq 2]` =

`z * z [z=y * y] [y=2 * 2]` =

`z * z [z=y * y] [y=4]` =



`z * z [z=4 * 4]` =

`16 * 16` =

256

Call-by-need: sometimes it is cool to have it

Example (Avoid heavy computation)

```
fst (x,_) = x
fst (1, 21010) = 1 -- 21010 not needed
```

Example (Avoid infinite loops ($\perp$))

```
myelem _ [] = False
myelem e (x:xs) = if (e==x) then True
                  else myelem e xs

neverEndingStory n = (-1) ^ n +
                    neverEndingStory (n-1)

Main> myelem 2 [1, 2, neverEndingStory 3, 4]
True
```

Call-by-need - a not-so-cool example

Example (Sometimes, we can't escape our fate)

`sumacc a [] = a`

`sumacc a (x:xs) = sumacc (a+x) xs`

`sumacc 0 [1,2,3] =`

`sumacc (0+1) [2,3] =`

`sumacc ((0+1)+2) [3] =`

`sumacc (((0+1)+2)+3) [] =`

`((0+1)+2)+3 =`

`(1+2)+3 = ...`

Call-by-need and strict application

Note

In this example, the **whole accumulator's expression** is built **before** being evaluated. This is an issue for a 10M element list.

Forcing argument evaluation

`$!` forces **strict** (call-by-value style) evaluation of the argument

Example (Summing up numbers)

```
sumacc a [] = a
```

```
sumacc a (x:xs) = (sumacc $! (a+x)) xs
```

Call-by-need - strict application

Example (Summing up numbers)

`sumacc a [] = a`

`sumacc a (x:xs) = (sumacc $! (a+x)) xs`

`sumacc 0 [1,2,3] =`

`(sumacc $! (0+1)) [2,3] =`

`(sumacc $! 1) [2,3] =`

`sumacc 1 [2,3] =`

`(sumacc $! (1+2)) [3] =`

`sumacc 3 [3] = ...`

Call-by-need - strict application

n	time spent (ms.)	digits of $n!$
5,000	31	16,327
50,000	1,575	213,238
500,000	204,696	2,632,343
1,000,000	840,458	5,565,710
2,000,000	3,500,371	11,733,841
3,000,000	7,757,876	18,128,485
5,000,000	21,295,103	31,323,383

Table: Computing $n!$ for different values of n

Machine: ASUS Notebook X551MA (Intel Celeron N2920 @1.86GHz, 4GiB RAM)

Call-by-need - strict application

Lazy vs. strict:

50,000!	no optimization	optimization (-O2)
lazy	2,728	1,083
strict (\$!)	1,089	1,082

Table: Time (ms.) spent to compute 50,000!

Machine: ASUS Notebook X551MA (Intel Celeron N2920 @1.86GHz, 4GiB RAM)

Defining lists

Definition

A list is either empty, or is an ordered pair **made** of an element ("head") and a list ("tail").

Remark

This definition shows the connection between the way we **represent** data and the way we **process** it.

Constructors

Example (Haskell operator :)

```
Prelude> 1:2:3:[]  
[1,2,3]
```

Remark

The cons operator is right associative.

Example (Haskell operator :)

```
Prelude> 1:2:[3]  
[1,2,3]
```

List processing: Haskell `maxl`

Example (Partial function)

```
Main> :set -W
```

```
Main> :{  
Main| maxl [x] = x  
Main| maxl (x:y:ys) = if (x>y) then maxl (x:ys)  
Main|                      else maxl (y:ys)  
Main| :}
```

```
<interactive>:9:1: warning:  
                    [-Wincomplete-patterns]  
Pattern match(es) are non-exhaustive  
In an equation for 'max':  
                    Patterns not matched: []
```

List processing: Haskell `length` and `append`

Example

```
my_length [] = 0
my_length (_:xs) = 1 + my_length xs
```

Example

```
infix 4 +++
[] +++ ys = ys
(x:xs) +++ ys = x:(xs +++ ys)
```

List processing: Haskell `take` and `drop`

Example

```
my_take 0 _      = []  
my_take k []     = []  
my_take k (x:xs) = x:my_take (k-1) xs
```

Example

```
my_drop 0 xs     = xs  
my_drop k []     = []  
my_drop k (x:xs) = my_drop (k-1) xs
```

List processing: Haskell reverse

Example

```
my_rev0 [] = []  
my_rev0 (x:xs) = my_rev0 xs ++ [x]
```

or:

Example

```
my_rev1 xs = my_rev1acc xs []  
  where  
    my_rev1acc [] ys = ys  
    my_rev1acc (x:xs) ys = my_rev1acc xs (x:ys)
```

Haskell `sum` and `minimum`

Example (Summing up a list)

```
Prelude> sum [2,3,4]  
9
```

Example (The lowest value in a list)

```
Prelude> minimum [2,1,3]  
1
```

Composed lists: flattening lists

Example

```
flatten [] = []  
flatten (x:xs) = x ++ flatten xs
```

Example

```
Main> flatten [ ["RO"], ["CH", "IL"], ["AD"] ]  
["RO","CH","IL","AD"]
```

List membership

Example (Haskell `myelem`, officially `elem`)

```
myelem e [] = False
myelem e (x:xs) | e == x = True
                  | otherwise = myelem e xs
```

Remark

`myelem` is written using **guarded equations**:

1. the test `e == x` is a guard; it is followed by `=` and the value the function should return if the guard test is true
2. the `|` symbol means "such that" and separates guards
3. the guard `otherwise` means `True` (a default guard)
4. (a matter of style): list names ought to end in `s`

Composed lists: `zip` and `unzip`

Example (`zip`)

```
myzip ([], []) = []
```

```
myzip (x:xs, y:ys) = (x,y):myzip(xs,ys)
```

```
Main> myzip (["RO", "CH", "IL"], [1,2,3])  
[("RO",1), ("CH",2), ("IL",3)]
```

Example (`unzip`)

```
myunzip [] = ([], [])
```

```
myunzip ((x,y):zs) =  
  let  
    (xs,ys) = myunzip zs  
  in  
    (x:xs,y:ys)
```

Composed lists: Haskell `zipWith` and `map`

Example (A higher order function: `zipWith`)

```
Prelude> zipWith (*) [1,2,3] [4,5,6]  
[4,10,18]
```

Remark

`zipWith` pairs up corresponding elements, then performs the specified operation (here, `*`) over each pair of components

Composed lists: Haskell `zipWith` and `map`

Example (Another higher order function: `map`)

```
Main> map (2*) [1000,2000,3000]  
[2000,4000,6000]
```

Remark

`map` applies a function given as an argument (here, `(2*)`) on each element of a list

DIY

How can I double each value in a matrix given as a list of lists?

Association lists

Definition

Association list = a list of pairs (*key*, *value*) with *key* of type Eq .

Example (Association lists in Haskell)

```
capitals =  
    [ ("NO", "Oslo"), ("FR", "Paris"), ("CH", "Bern") ]
```

```
assoc (x, []) = []  
assoc (x, (y,z):lp) = if x == y then [z]  
                      else assoc (x, lp)
```

Example

```
Main> assoc ("CH", capitals)  
["Bern"]
```

Summary

- ▶ curried functions call one argument at a time and are a powerful mechanism in FP
- ▶ function could be regarded as a type
- ▶ recursive types are possible
- ▶ call-by-need (arguments evaluated at most once, when needed) and call-by-value (arguments evaluated before being processed) have both advantages and disadvantages
- ▶ list processing is based on recursivity + pattern matching, in line with the definition of the list